NOVA FEST 2027

Instrucciones generales

- Crea index.html, scss/estilos.scss y css/estilos.css. En el ejercicio 6 organizarás el SCSS en archivos parciales.
- No modifiques manualmente css/estilos.css: debe generarse al compilar Sass.
- Utiliza el HTML facilitado. No es necesario modificarlo en los ejercicios.
- Comprueba la página en el navegador al finalizar cada ejercicio. Los estilos de ejercicios anteriores deben conservarse, salvo cuando se indique expresamente su sustitución.

Estructura inicial

```
nova-fest/
   index.html
   scss/
       estilos.scss
    css/
       estilos.css
```

HTML inicial (no modificar)

- Html proporcionado junto al enunciado

Ejercicio 1. Variables y operaciones

Objetivo: Crear una cabecera y una portada atractivas desde el primer ejercicio, usando variables y cálculos de Sass.

- Declara en estilos.scss estas variables: $fondo: #101321; $superficie: #1d2235; $texto: #f7f7fb; $acento: #d7ff4b; $secundario: #ff6b91; $espacio: 8px; $radio: 16px; $ancho: 1100px.
- Aplica box-sizing: border-box al selector universal *. En body elimina el margen, establece el fondo $fondo, el color $texto y font-family: Arial, sans-serif.
- Configura .contenedor con width: 90%, max-width: $ancho y margin: auto.
- En .cabecera aplica fondo $superficie y padding: $espacio * 2. Organiza .cabecera__interior mediante Flexbox, justify-content: space-between y align-items: center.
- En .logo elimina el subrayado, usa color $texto, font-weight: bold y font-size: 24px. Colorea su span con $acento. En .menu aplica display: flex y gap: $espacio * 3; los enlaces serán blancos y sin subrayado.
- En .portada aplica un fondo con linear-gradient(135deg, $superficie, $fondo), padding: $espacio * 10 0 y min-height: 420px. En .portada h1 utiliza font-size: 64px y margin: 20px 0. Colorea su span con $acento; en .portada__fecha utiliza color $secundario y font-weight: bold.
- Da a .boton display: inline-block, padding: $espacio * 2 $espacio * 3, border-radius: $radio, fondo $acento, color $fondo, font-weight: bold y text-decoration: none.

- En .seccion aplica padding: $espacio * 7 0; en .pie utiliza fondo $superficie, texto centrado y padding: $espacio * 3. Añade una regla general para .seccion h2 con margin-top: 0.
- Comprueba en el CSS compilado que las operaciones se han convertido en medidas concretas (por ejemplo, 16px, 24px y 80px).

Comprobación: La cabecera y la portada deben estar maquetadas y resultar visualmente atractivas. Las secciones inferiores todavía pueden aparecer sin formato específico.

Ejercicio 2. Anidamiento de selectores y &

Objetivo: Diseñar la sección de artistas y practicar la anidación de selectores, pseudoclases y modificadores.

- Reorganiza los estilos de .cabecera, .logo y .portada mediante anidamiento. En .logo utiliza & span o anidamiento span; en .portada anida h1 y su span.
- Dentro de .menu anida a y dentro de a añade &:hover para cambiar el color a $acento.
- En .artistas utiliza display: grid, grid-template-columns: repeat(3, 1fr) y gap: $espacio * 3.
- Crea .artista con fondo $superficie, padding: $espacio * 3 y border-radius: $radio. Anida h3 (margin-bottom: 8px) y p (margin-bottom: 0).
- Dentro de .artista define &__imagen con display: flex, justify-content: center, align-items: center, height: 160px, font-size: 48px, font-weight: bold, fondo $fondo y border-radius: $radio.
- Dentro de .artista define &--destacado con border: 2px solid $acento. Añade &:hover en .artista para aplicar box-shadow: 0 6px 20px rgba(0, 0, 0, .3).
- Da a .genero font-size: 13px y font-weight: bold. Todavía no es necesario diferenciar los colores de cada género.

Comprobación: Las tres tarjetas aparecen en columnas; la primera tiene un borde destacado y al pasar el ratón se aprecia una sombra. El CSS generado contiene .artista__imagen y .artista--destacado.

Ejercicio 3. Listas, mapas e interpolación

Objetivo: Usar colecciones Sass para asignar tamaños y preparar estilos por género musical.

- Añade @use "sass:map"; y @use "sass:list"; al principio de estilos.scss.
- Define el mapa $generos: (electronica: #d7ff4b, rock: #ff6b91, pop: #8dc8ff).
- Define la lista $tamanos: 14px, 20px, 32px. Utiliza list.nth($tamanos, 2) para el tamaño de .artista h3 y list.nth($tamanos, 3) para el tamaño de .seccion h2.
- Define $genero-activo: electronica; y genera la clase .genero--#{$genero-activo} mediante interpolación. Su color será map.get($generos, $genero-activo).
- Utiliza map.get($generos, rock) para establecer el color del texto de .entrada__precio (se verá al maquetar las entradas en el ejercicio 4).
- No escribas manualmente el nombre de la clase .genero--electronica: debe generarse por interpolación. Las otras dos variantes se generarán en el ejercicio 7.

Comprobación: El género Electrónica aparece en verde lima, los títulos de las tarjetas miden 20px y los títulos de sección 32px.

Ejercicio 4. Mixins con parámetros

Objetivo: Reutilizar estilos de botones y completar la sección de entradas.

- Crea @mixin boton-estilo($fondo, $texto: #101321) con background-color: $fondo; color: $texto; padding: 16px 24px; border: 0; border-radius: $radio. El segundo parámetro debe tener un valor por defecto legible.
- En .boton conserva display: inline-block, font-weight: bold y text-decoration: none. Sustituye sus propiedades repetidas de color, relleno y borde por @include boton-estilo($acento, $fondo).
- Dentro de .boton crea &--principal y aplícale @include boton-estilo($secundario, $fondo). Añade &:hover con filter: brightness(.9) a .boton.
- Crea un mixin sin parámetros @mixin centrar-flex con display: flex, justify-content: center y align-items: center. Utilízalo en .artista__imagen en lugar de repetir esas tres declaraciones.
- En .entradas utiliza background: $superficie. Dentro, anida &__lista con display: grid, grid-template-columns: repeat(2, 1fr) y gap: 24px.
- Crea .entrada con fondo $fondo, padding: 32px y border-radius: $radio. Anida &__precio con font-size: 32px y font-weight: bold; anida &--vip con border: 2px solid $secundario.
- Comprueba que los botones generales y los principales se distinguen visualmente y que las tarjetas de entradas se muestran en dos columnas.

Comprobación: La sección Entradas queda maquetada; la tarjeta VIP está destacada y el código reutiliza mixins en lugar de copiar declaraciones.

Ejercicio 5. Funciones, @extend y placeholders

Objetivo: Aplicar cálculos reutilizables y estilos comunes a los avisos informativos.

- Importa @use "sass:math"; al principio del archivo.
- Crea @function px-a-rem($px, $base: 16px) { @return math.div($px, $base) * 1rem; }.
- Utiliza px-a-rem(64px) en .portada h1, px-a-rem(32px) en .seccion h2 y px-a-rem(20px) en .artista h3, sustituyendo sus tamaños anteriores.
- Define %aviso-base con padding: 16px; border-radius: 8px; margin-bottom: 12px; font-weight: bold.
- Crea .aviso--info, .aviso--exito y .aviso--importante. En las tres utiliza @extend %aviso-base. Asigna respectivamente background-color: #dbeafe, #dcfce7 y #ffe4e6; y color: #1e40af, #166534 y #9f1239.
- Comprueba que el placeholder no aparece como selector independiente en el CSS generado y que la función produce 4rem, 2rem y 1.25rem.

Comprobación: Los tres avisos tienen la misma estructura visual pero colores distintos; los títulos siguen viéndose igual tras pasar sus tamaños a rem.

Ejercicio 6. Archivos parciales, @use y @forward

Objetivo: Organizar el proyecto Sass sin cambiar el resultado visual.

- Crea la carpeta scss/tema/ y dentro los archivos _variables.scss, _mixins.scss, _funciones.scss y _index.scss.
- Traslada todas las variables globales (incluido el mapa $generos y la lista $tamanos) a _variables.scss; los mixins a _mixins.scss; y la función px-a-rem a _funciones.scss.
- En _mixins.scss utiliza @use "variables" as v; para acceder a las variables necesarias. En _funciones.scss conserva @use "sass:math";.
- En _index.scss escribe @forward "variables"; @forward "mixins"; y @forward "funciones";.
- En estilos.scss incorpora @use "tema" as t; y conserva @use "sass:map"; y @use "sass:list"; cuando correspondan.
- Actualiza todas las referencias: $fondo pasa a t.$fondo, @include boton-estilo(...) a @include t.boton-estilo(...), y px-a-rem(...) a t.px-a-rem(...). También debes actualizar las referencias a listas y mapas.
- Compila y compara la página antes y después. No debe cambiar su apariencia.

Comprobación: El proyecto compila sin errores y el archivo estilos.scss ya no contiene definiciones de variables globales, mixins ni funciones.

Ejercicio 7. Condicionales, bucles y responsive

Objetivo: Automatizar la generación de clases y adaptar la página a móviles.

- Elimina la regla interpolada individual .genero--electronica creada en el ejercicio 3. Recorre t.$generos con @each $nombre, $color y genera .genero--#{$nombre} con color: $color. Deben funcionar los tres géneros del HTML.
- Utiliza @for $i from 1 through 4 para generar .espacio-#{$i} con margin-top: $i * t.$espacio. No escribas manualmente las cuatro clases.
- En _mixins.scss crea @mixin aspecto($modo) con tres ramas: si $modo == oscuro, fondo v.$fondo y texto v.$texto; si $modo == claro, fondo blanco y texto v.$fondo; en cualquier otro caso, @warn "Modo desconocido" y fondo v.$superficie y texto v.$texto.
- Aplica @include t.aspecto(oscuro) a .pie. Prueba temporalmente otro valor para observar la advertencia y restaura oscuro.
- Añade @media (max-width: 760px) al final de estilos.scss. Dentro: .cabecera__interior { flex-direction: column; gap: 16px; }; .menu { flex-wrap: wrap; justify-content: center; }; .portada { padding: 48px 0; min-height: auto; }; .portada h1 { font-size: 40px; }; .artistas y .entradas__lista { grid-template-columns: 1fr; }.
- Comprueba la web a 760px y a 390px. No deben aparecer tarjetas cortadas ni desplazamiento horizontal.

Comprobación: Los tres géneros tienen colores distintos, se generan cuatro clases de espaciado, el pie utiliza el mixin condicional y la web se adapta a móvil.